MACHINE LEARNING

UNIT – V : Clustering

Easy digital notes with diagrams and solved examples (JNTU B.Tech, 23A31401T)

Topics in this unit

1. Introduction to clustering, clustering of patterns, partitioning of data
2. Matrix factorization
3. Hierarchical: divisive and agglomerative clustering
4. Partitional clustering and K-Means
5. Soft partitioning, soft clustering, Fuzzy C-Means
6. Rough clustering and Rough K-Means
7. Expectation Maximization (EM) based clustering
8. Spectral clustering

> * **Study tip:**
> All numbers in the worked examples were computed by a program, so you can copy them into an exam answer. Learn the RED words, then redo each small example once on paper.

Syllabus (as given)

Clustering: Introduction to Clustering, Partitioning of Data, Matrix Factorization, Clustering of Patterns, Divisive Clustering, Agglomerative Clustering, Partitional Clustering, K-Means Clustering, Soft Partitioning, Soft Clustering, Fuzzy C-Means Clustering, Rough Clustering, Rough K-Means Clustering Algorithm, Expectation Maximization-Based Clustering, Spectral Clustering.

Part I : Foundations of Clustering

5.1 Introduction to Clustering

Clustering groups a set of objects so that objects in the **same group (cluster)** are **similar** to each other and **different** from objects in other groups. It is **unsupervised learning**: there are **no class labels**, the structure is found from the data alone.

- **Goal:** high **intra-cluster similarity** (compact clusters) and low **inter-cluster similarity** (well separated clusters).
- **Applications:** customer segmentation, document / news grouping, image segmentation, gene-expression analysis, anomaly detection, data compression (vector quantization).
- **Clustering vs classification:** classification learns from labelled examples and predicts a known class; clustering **discovers** the groups itself and the number of groups may be unknown.

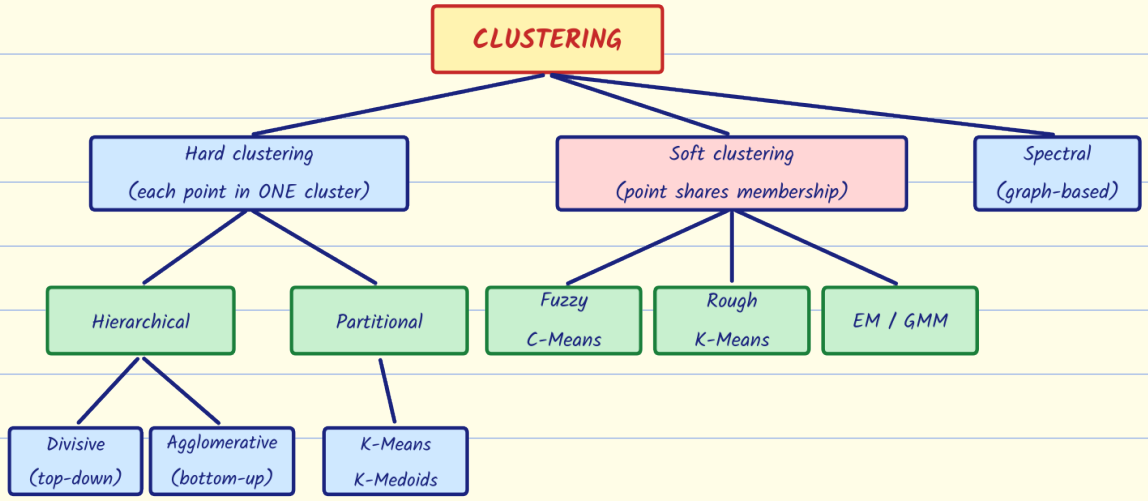


Fig 5.1 : Types of clustering covered in this unit

> * **Remember:**
> Clustering has no single 'correct' answer. The result depends on the distance measure, the algorithm and the number of clusters chosen.

5.2 Clustering of Patterns

A **pattern** is one object described by **d features**, written as a vector $x = (x_1, \ldots, x_d)$. A data set of n patterns is the **n × d pattern matrix**. Clustering of patterns needs a way to measure how close two patterns are.

Measure	Formula	Use
Euclidean	$d(x,y) = \sqrt{\Sigma (x_i - y_i)^2}$	Default for numeric data; K-Means
Manhattan	$d(x,y) = \Sigma \lvert x_i - y_i \rvert$	Robust to outliers, grid-like data
Minkowski	$d = (\Sigma \lvert x_i - y_i \rvert^p)^{1/p}$	$p = 1$ Manhattan, $p = 2$ Euclidean
Cosine similarity	$\cos \theta = (x \cdot y) / (\lvert x \rvert \lvert y \rvert)$	Text / document vectors
Hamming / Jaccard	Count of differing bits / $\lvert A \cap B \rvert$ / $\lvert A \cup B \rvert$	Binary and set data

Steps of a clustering task

1. **Represent** patterns as feature vectors (scale features so that no feature dominates).
2. Choose a **proximity measure** (distance or similarity).
3. Choose the **clustering algorithm** and the number of clusters K (if needed).
4. **Validate** the result: **internal** measures (SSE, silhouette) need no labels; **external** measures (purity, Rand index) compare with known labels.
5. **Interpret** the clusters (give them meaning, e.g. 'high-spending young customers').

> * **Study tip:**
> ALWAYS standardise features first. If income is in lakhs and age in years, income will dominate the Euclidean distance.

5.3 Partitioning of Data

A **partition** divides data $X = \{x_1, ..., x_n\}$ into K clusters $C_1, ..., C_K$. A **hard (crisp) partition** must satisfy three conditions:

- Every cluster is **non-empty**: $C_i \neq \emptyset$
- Clusters **cover** all data: $C_1 \cup C_2 \cup ... \cup C_K = X$
- Clusters are **disjoint**: $C_i \cap C_j = \emptyset$ for $i \neq j$ (each object in exactly ONE cluster).

In a **soft partition** the third condition is relaxed. Each object has a **membership degree** $u_{ij} \in [0,1]$ in every cluster, with $\Sigma_j u_{ij} = 1$. A hard partition is the special case where every u_{ij} is 0 or 1.

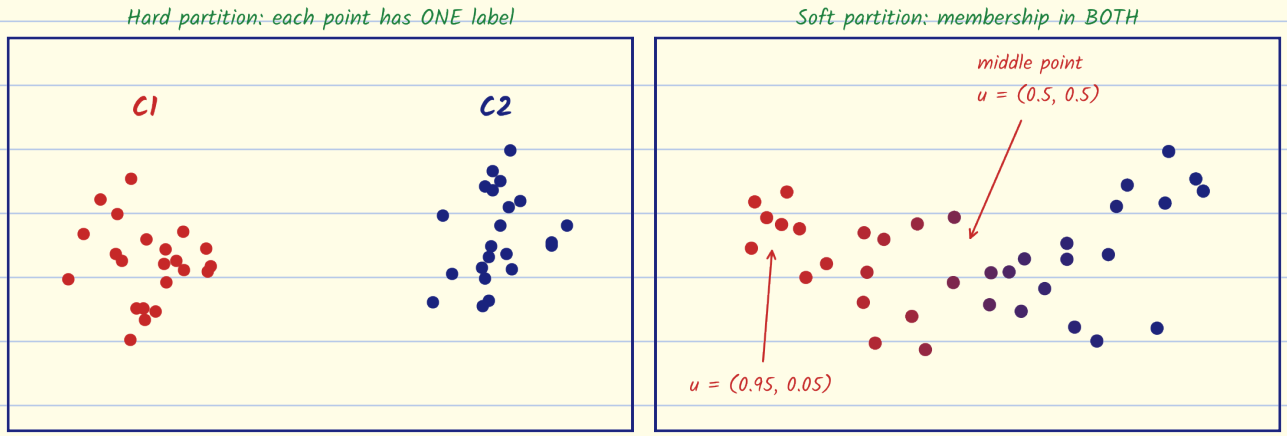
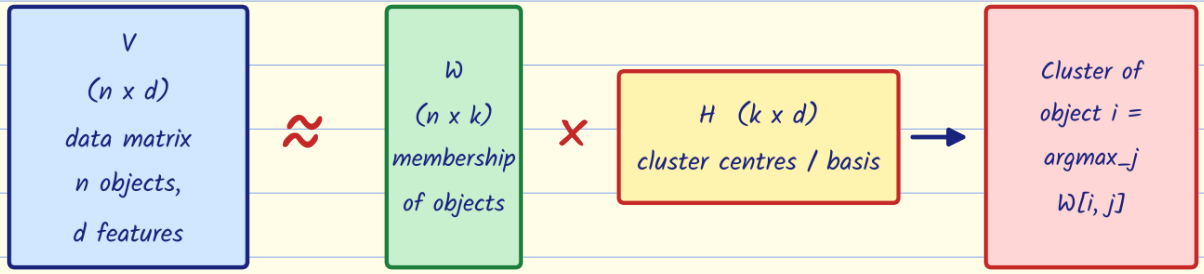


Fig 5.2 : Hard partition and soft partition of the same kind of data

Why we need smart algorithms: the number of ways to partition n objects into K non-empty clusters is the **Stirling number of the second kind** $S(n,K) \approx K^n / K!$. Check: $S(6,2) = 31$; $S(10,3) = 9,330$; $S(100,5) \approx 6.57e+67$. Trying every partition is impossible, so we use **heuristics** (K-Means) that find a good, not always the best, partition.

5.4 Matrix Factorization

Matrix factorization approximates the data matrix V ($n \times d$) by the product of two smaller matrices: $V \approx W \cdot H$ with W ($n \times k$) and H ($k \times d$), k much smaller than n and d. It gives a **low-rank** view of the data and can be read as clustering.



k << n, d : each row of W says which cluster, each row of H is one centre

Fig 5.3 : Matrix factorization view of clustering

Method	Constraint	Clustering meaning
K-Means	W is binary, exactly one 1 per row	H = cluster centres; W = hard assignment. Minimises $\|\|V - WH\|\|^2$ (same as SSE)
NMF	V, W, H all ≥ 0	Parts-based; W gives soft membership; cluster = argmax of row of W
SVD / PCA	$V = U\Sigma V^T$, orthogonal factors	Keep top k singular vectors, then cluster rows of $U\Sigma$ (denoise + reduce dimension)
Spectral	Eigenvectors of Laplacian	Factorise the graph matrix, cluster the eigenvector rows (see 5.14)

NMF objective: minimise $\|\|V - WH\|\|_F^2$ subject to W, H ≥ 0. Lee-Seung multiplicative updates: $H \leftarrow H \odot (W^TV) / (W^TWH)$, $W \leftarrow W \odot (VH^T) / (WHH^T)$.

Tiny example: 4 documents × 4 words (counts). NMF with k = 2 gives the memberships below: D1, D2 load on factor 2 (words w1, w2) and D3, D4 on factor 1 (words w3, w4), so the documents form **2 clusters**.

Doc	w1	w2	w3	w4	W (k=2)	Cluster
D1	3	2	0	0	(0.00, 1.60)	B
D2	2	3	0	1	(0.14, 1.62)	B
D3	0	0	4	3	(1.84, 0.00)	A
D4	1	0	3	4	(1.83, 0.26)	A

Part II : Hierarchical Clustering

5.5 Divisive Clustering (top-down)

Hierarchical clustering builds a **tree of clusters** (dendrogram) and needs no K in advance. **Divisive** clustering starts with **all objects in one cluster** and **splits** it again and again until every object is alone (or K clusters remain). **Agglomerative** does the opposite.

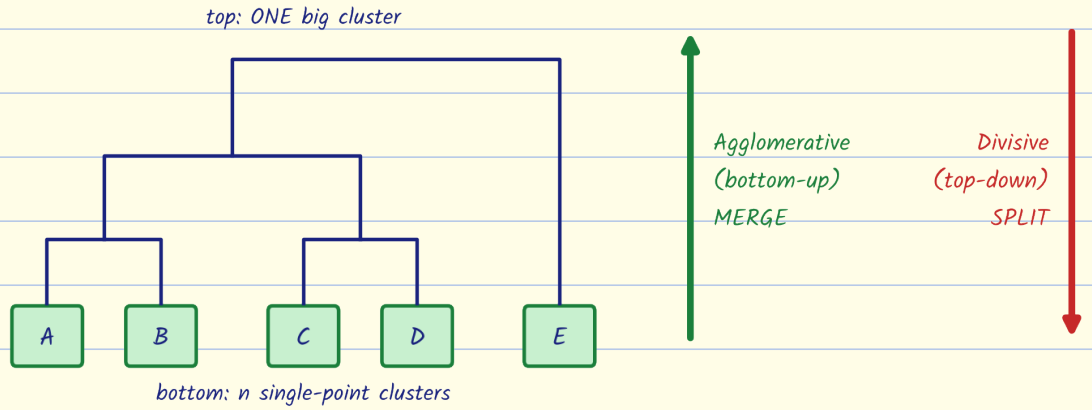


Fig 5.4 : Divisive (top-down) and agglomerative (bottom-up) directions

DIANA algorithm (Divisive Analysis)

1. Put all objects in one cluster. Pick the cluster with the **largest diameter** to split.
2. Find the object with the **largest average dissimilarity** to the others - it starts the **splinter group**.
3. For each remaining object compute avg distance to main group - avg distance to splinter group. If it is **positive**, move the object to the splinter group. Repeat until no object moves.
4. Repeat from step 1 until clusters are single objects or K clusters are reached.

Example (1-D points A=1, B=2, C=6, D=8, E=15). Average dissimilarity of each point to the rest: A 6.75, B 6.00, C 5.00, D 5.50, E 10.75. E is largest, so splinter = {E}; the other points are closer to their own group, so nobody moves: split 1 = {A,B,C,D} | {E}. Largest-diameter cluster is {A,B,C,D} (diameter 7): averages A 4.33, B 3.67, C 3.67, D 5.00, so splinter starts with D; C is closer to D (avg 2) than to A,B (avg 4.5), so it moves. Split 2 = {A,B} | {C,D}. Final tree: same as the agglomerative tree of 5.6.

- **Bisecting K-Means:** a popular practical divisive method - repeatedly split the cluster with the largest SSE using K-Means with K = 2.
- **Cost:** there are $2^{n-1} - 1$ ways to split n objects, so exact divisive clustering is very costly; heuristics (DIANA) are used. Good at finding the **few large clusters** first.

5.6 Agglomerative Clustering (bottom-up)

Agglomerative (AGNES) starts with **n clusters of one object each** and **merges** the two **closest** clusters in every step until a single cluster remains. The history of merges is drawn as a **dendrogram**; cutting it at a height gives the clusters.

1. Compute the n x n distance matrix; every object is its own cluster
2. Find the two closest clusters Ci, Cj
3. Merge them; update the distances to the new cluster (linkage rule)
4. Repeat 2-3 until one cluster is left (or K clusters remain)

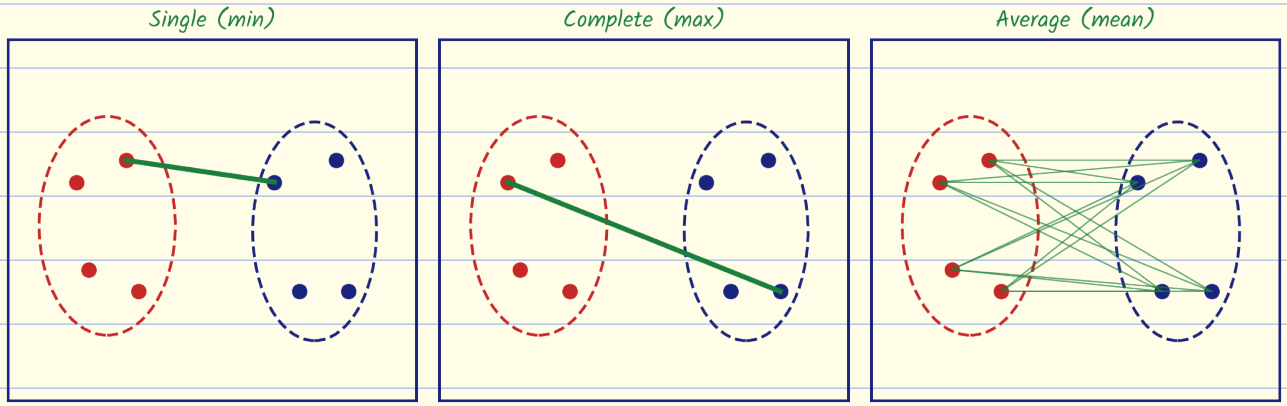


Fig 5.5 : Single, complete and average linkage

Linkage	Distance between clusters A and B	Behaviour
Single	min d(a,b) (closest pair)	Finds long, chain-like shapes; suffers **chaining**
Complete	max d(a,b) (farthest pair)	Compact, equal-sized clusters; sensitive to outliers
Average	mean of all d(a,b) (UPGMA)	Compromise between single and complete
Centroid / Ward	distance of centres / increase in SSE	Ward gives compact, balanced clusters

Worked example: points A=1, B=2, C=6, D=8, E=15 (1-D, so d = |x - y|).

d	A	B	C	D	E
A	0	1	5	7	14
B	1	0	4	6	13
C	5	4	0	2	9
D	7	6	2	0	7
E	14	13	9	7	0

Step	Merge (single linkage)	Distance	Clusters left
1	A + B	1	{AB}, C, D, E
2	C + D	2	{AB}, {CD}, E
3	{AB} + {CD}	min(|2-6|) = 4	{ABCD}, E
4	{ABCD} + E	min(|8-15|) = 7	{ABCDE}

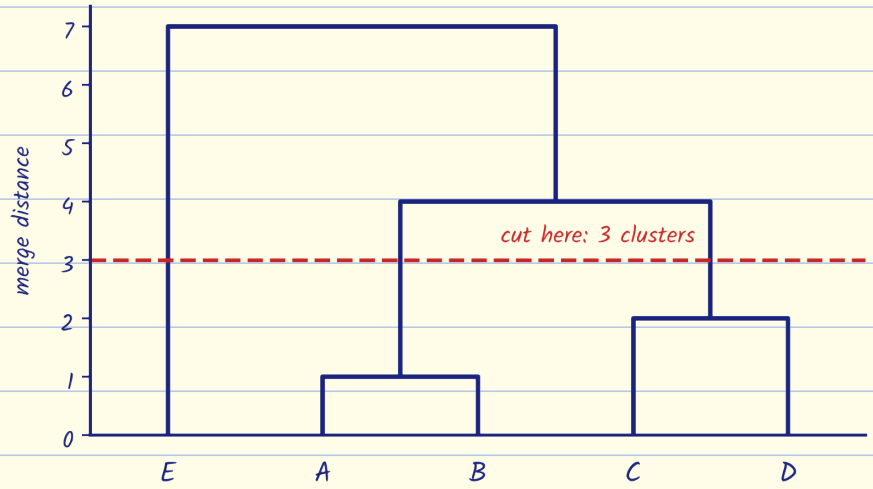


Fig 5.6 : Dendrogram of the single-linkage example; cutting at height 3 gives {A,B}, {C,D}, {E}

Same data, complete linkage: steps 1, 2 are the same (1, 2); step 3 merges {AB} and {CD} at max distance 7 (not 4); step 4 adds E at 14. Average linkage gives 5.5 and 10.75. So the **linkage choice changes the heights**, though here the tree shape stays the same.

- **Choosing K:** cut the dendrogram where there is the **longest vertical gap** (a big jump in merge distance) - here the jump 2 → 4 → 7.
- **Complexity:** $O(n^2)$ memory and $O(n^3)$ time ($O(n^2 \log n)$ with a heap). Hence **not suitable for very large n**.
- **Pros:** no K needed in advance, dendrogram is informative, any distance works. **Cons:** a merge can never be undone (greedy), slow, sensitive to noise.

Part III : Partitional Clustering and K-Means

5.7 Partitional Clustering

Partitional clustering directly divides the data into **K flat (non-overlapping) clusters** in one go, then **improves** the partition step by step by optimising a criterion (usually minimise the squared error). K must be given. Examples: K-Means, K-Medoids (PAM), K-Modes, CLARA.

Point	Hierarchical	Partitional
Output	Tree (dendrogram), nested clusters	One flat partition into K clusters
K needed?	No (cut the tree later)	Yes (input)
Time	$O(n^2)$ - $O(n^3)$	$O(n \cdot K \cdot t)$ - fast
Undo a step?	No (greedy merges / splits)	Yes - points can move between clusters
Best for	Small data, taxonomy	Large data, round clusters

5.8 K-Means Clustering

K-Means (MacQueen, Lloyd) represents each cluster by its **centroid (mean)**. It alternates between **assigning** points to the nearest centroid and **moving** each centroid to the mean of its points.

Objective (SSE / WCSS): $J = \sum_{j=1..K} \sum_{x \in C_j} || x - \mu_j ||^2$

Assign: $x \rightarrow C_j$ with smallest $||x - \mu_j||$ **Update:** $\mu_j = (1/|C_j|) \sum_{x \in C_j} x$

1. Choose K and initial centroids mu_1 ... mu_K (random points / k-means++)
2. ASSIGN each point to its nearest centroid
3. UPDATE each centroid = mean of the points assigned to it
4. Repeat 2-3 until assignments stop changing (or J stops decreasing)

Worked example (K = 2). Points P1(1,1), P2(2,1), P3(1,2), P4(5,4), P5(6,5), P6(5,5). Take **initial centroids = P1 and P2**.

Iteration 1: c1 = (1.00, 1.00), c2 = (2.00, 1.00). Assign: C1 = {P1, P3}, C2 = {P2, P4, P5, P6}. **New centroids:** c1 = (1.00, 1.50), c2 = (4.50, 3.75); SSE = 20.25.

Iteration 2: c1 = (1.00, 1.50), c2 = (4.50, 3.75). Assign: C1 = {P1, P2, P3}, C2 = {P4, P5, P6}. **New centroids:** c1 = (1.33, 1.33), c2 = (5.33, 4.67); SSE = 2.67.

Point	It.1: d(c1)	d(c2)	Cluster	It.2: d(c1)	d(c2)	Cluster
P1	0.00	1.00	C1	0.50	4.45	C1
P2	1.00	0.00	C2	1.12	3.72	C1
P3	1.00	1.41	C1	0.50	3.91	C1
P4	5.00	4.24	C2	4.72	0.56	C2
P5	6.40	5.66	C2	6.10	1.95	C2
P6	5.66	5.00	C2	5.32	1.35	C2

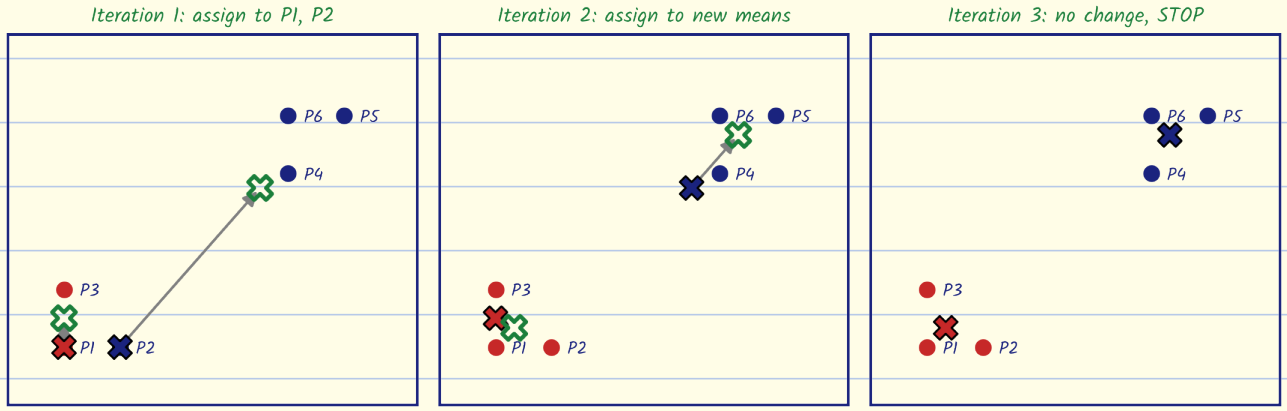


Fig 5.7 : K-Means on the six points: assign, move the centroids, stop when nothing changes

Iteration 3 (check): with c1 = (1.33, 1.33) and c2 = (5.33, 4.67) the assignment is the same, so the algorithm **has converged**. Final SSE = 2.67 (it fell from 20.25 to 2.67 to 2.67; J never increases). Silhouette score of this clustering = 0.78.

Choosing K, initialisation and variants

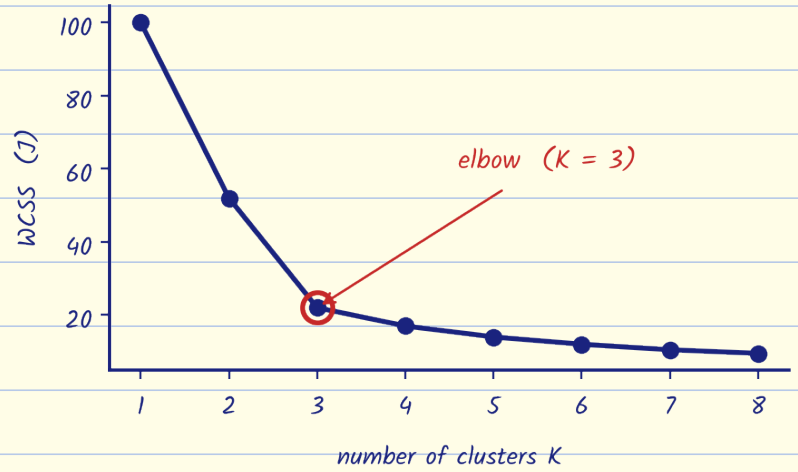


Fig 5.8 : Elbow method (illustrative values): WCSS falls sharply until K = 3, then slowly

- **Elbow method:** plot J against K; pick the K at the bend. **Silhouette:** $s(i) = (b - a) / \max(a, b)$, a = mean distance to own cluster, b = mean distance to nearest other cluster; s near +1 is good; choose K with highest average s.
- **K-Means++:** choose the first centroid at random, each next one with probability $\propto D(x)^2$ (distance to nearest chosen centroid) - spread-out start, better results. Run several times, keep the lowest J.
- **K-Medoids (PAM):** centre is an actual data point (medoid) - robust to outliers, works with any distance. **Mini-batch K-Means:** update with small random batches for big data.
- **Complexity:** $O(n \cdot K \cdot d \cdot t)$ for t iterations - linear in n. **Always converges** to a **local** minimum (not necessarily the global one).

Pros	Cons
Simple, fast, scales to large data; easy to implement	K must be chosen beforehand
Compact clusters; works well for round, equal-sized blobs	Depends on initial centres - local minimum
Output is easy to interpret (centroids)	Fails on non-convex shapes, unequal sizes / densities
Many variants (K-Medoids, mini-batch)	Sensitive to outliers and to feature scaling; hard assignment only

Part IV : Soft Clustering Methods

5.9 Soft Partitioning and Soft Clustering

Soft clustering (overlapping clustering) lets one object belong to **several clusters** with a **degree**. The result is a **membership matrix U** ($n \times K$) instead of one label per point. This is natural for borderline objects: a document on 'sports finance', a pixel on an object edge, a gene with two functions.

Point	Hard clustering	Soft clustering
Membership	$u_{ij} \in \{0, 1\}$	$u_{ij} \in [0, 1]$ (fuzzy / probability) or lower / upper sets (rough)
Row sum	$\Sigma_j u_{ij} = 1$ (single 1)	$\Sigma_j u_{ij} = 1$ (shared) [rough: sets]
Boundary points	Forced into one cluster	Shown as ambiguous - extra information
Examples	K-Means, Agglomerative	Fuzzy C-Means, Rough K-Means, EM / GMM
Cost	Cheaper, simpler	More computation; need a rule to get a final hard label (take the maximum u)

- Three ways to be soft: **fuzzy** (degrees in [0,1], FCM), **probabilistic** (posterior probabilities, EM/GMM), **rough** (lower and upper approximation sets).
- **Fuzzy vs probabilistic:** both rows sum to 1, but a probabilistic membership is a **probability under a model**, a fuzzy membership is just a **degree of belonging**.

5.10 Fuzzy C-Means (FCM) Clustering

Fuzzy C-Means (Dunn 1973, Bezdek 1981) is the **soft version of K-Means**. There are C clusters with centres v_j; each point x_i has a membership u_{ij} in each cluster, with $\Sigma_j u_{ij} = 1$.

Objective: $J_m = \Sigma_{i=1..n} \Sigma_{j=1..C} (u_{ij})^m \cdot || x_i - v_j ||^2$ $m > 1$ = fuzzifier
Membership: $u_{ij} = 1 / \Sigma_{k=1..C} (d_{ij} / d_{ik})^{2/(m-1)}$ $d_{ij} = ||x_i - v_j||$
Centre: $v_j = \Sigma_i (u_{ij})^m x_i / \Sigma_i (u_{ij})^m$

1. Fix C, fuzzifier m (usually 2), tolerance eps; initialise U randomly (rows sum to 1)
2. Compute centres v_j from U using the centre formula
3. Update memberships u_ij from the distances
4. Stop if max change in U < eps, else go to 2

- **Fuzzifier m:** $m \to 1$ gives hard K-Means ($u \to 0$ or 1); large m makes all memberships $\to 1/C$ (very fuzzy). $m = 2$ is the common choice.
- A point very close to a centre gets $u \approx 1$ for it; a point midway gets $u = 0.5, 0.5$ (for $C = 2$). Unlike probabilities, memberships do not use a density model.

Worked example (1-D, C = 2, m = 2). Data $x = 1, 2, 5, 8$; current centres $v_1 = 1.5$, $v_2 = 6.5$. For $x = 1$: $u_1 = 1 / (1 + (0.5/5.5)^2) = 0.9918$. All memberships:

x	d1	d2	u1	u2	u1²	u2²
1	0.5	5.5	0.9918	0.0082	0.9837	0.0001
2	0.5	4.5	0.9878	0.0122	0.9758	0.0001
5	3.5	1.5	0.1552	0.8448	0.0241	0.7137
8	6.5	1.5	0.0506	0.9494	0.0026	0.9014

New centres: $v_1 = \Sigma u_1^2 x / \Sigma u_1^2 = 1.5488$; $v_2 = 6.6736$. Objective J fell from 4.532 to 4.456. After more iterations the centres settle near 1.5 and 6.7. Point $x = 5$ stays mostly in cluster 2 ($u_2 = 0.81$) but keeps $u_1 = 0.19$ in cluster 1 - the soft information.

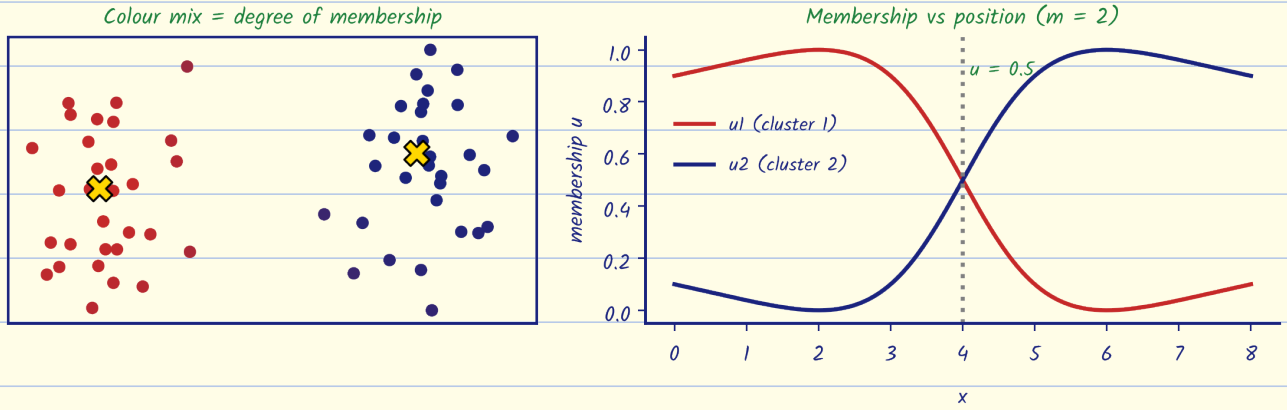


Fig 5.9 : FCM memberships: colour mix on the left, membership curves for two centres on the right

- **Pros:** gives degrees of belonging, handles overlapping clusters, usually more robust than K-Means near boundaries.
- **Cons:** C and m must be chosen; sensitive to initialisation and noise (memberships sum to 1, so outliers still get high membership); slower than K-Means; assumes round clusters.

5.11 Rough Clustering

Rough set theory (Pawlak, 1982) describes a set that cannot be defined exactly using **two crisp sets**: the **lower approximation** and the **upper approximation**. For a cluster C:

- **Lower approximation** $\underline{C}$: objects that **surely** belong to C (and to no other cluster).
- **Upper approximation** $\overline{C}$: objects that **possibly** belong to C (surely + doubtful ones).
- **Boundary region** = $\overline{C} - \underline{C}$: doubtful objects, shared with other clusters. Properties: $\underline{C} \subseteq C \subseteq \overline{C}$; if the boundary is empty the cluster is an ordinary crisp set.
- If an object is in the lower approximation of C_i it is in the lower approximation of **no other** cluster, and also in the upper approx. of C_j; an object in no lower approximation belongs to the upper approximation of **at least two** clusters.

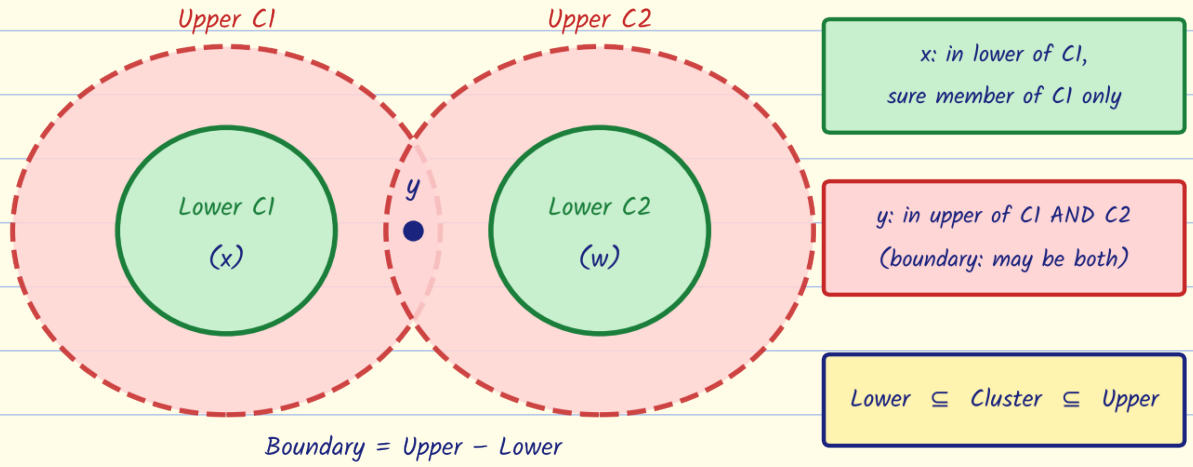


Fig 5.10 : Rough clustering: lower approximation (sure), upper approximation (possible) and boundary

Idea: unlike fuzzy sets (a number for every point), rough clustering gives a **sure core** plus a **doubtful fringe** - useful when we only want to say 'definitely in', 'maybe in'.

5.12 Rough K-Means Clustering Algorithm (Lingras and West)

Rough K-Means keeps the K-Means loop but every cluster has a **lower** and an **upper** approximation, and each centre is a **weighted mean** of the two parts.

1. Choose K, threshold eps, weights w_l + w_u = 1 (w_l > w_u), initial centres
2. For each x: find nearest centre m (d_min). T = { j : d(x,c_j) - d_min <= eps }
3. If |T| = 1 : x goes to LOWER and UPPER approximation of that cluster
   If |T| > 1 : x goes to the UPPER approximation of every cluster in T (boundary)
4. New centre: c_j = w_l * mean(lower_j) + w_u * mean(boundary_j)
   (if boundary_j is empty, c_j = mean(lower_j))
5. Repeat 2-4 until the centres stop changing

Centre update: $c_j = w_l \cdot (1/|\underline{C}_j|) \sum_{x \in \underline{C}_j} x + w_u \cdot (1/|\overline{C}_j - \underline{C}_j|) \sum_{x \in \overline{C}_j - \underline{C}_j} x$

Worked example: x = 1, 2, 5, 8, 9; c_1 = 1.5, c_2 = 8.5; ε = 1; w_l = 0.7, w_u = 0.3. Distances (to c_1, c_2): 1→(0.5, 7.5), 2→(0.5, 6.5), 5→(3.5, 3.5), 8→(6.5, 0.5), 9→(7.5, 0.5). Point 5 has difference 0 ≤ ε, so T = {1,2}: it is a boundary object. All others have a difference > 1, so they are sure members.

Cluster	Lower approx.	Boundary	Upper approx.	New centre
1	{1, 2} mean 1.5	{5}	{1, 2, 5}	0.7×1.5 + 0.3×5 = 2.55
2	{8, 9} mean 8.5	{5}	{5, 8, 9}	0.7×8.5 + 0.3×5 = 7.45

- **Effect:** the boundary point pulls each centre only with the small weight w_u, so uncertain points do not distort the clusters. With ε = 0 and w_u = 0 it behaves like K-Means.
- **Parameters:** ε controls how big the boundary is, w_l / w_u control the influence. **Pros:** handles vague boundaries and outliers better, no fuzzy-membership computation. **Cons:** three parameters (K, ε, w) to tune.

5.13 Expectation Maximization (EM) Based Clustering

EM clustering assumes the data come from a **mixture of K Gaussians** (GMM): $p(x) = \Sigma_k \pi_k N(x \mid \mu_k, \Sigma_k)$, with **mixing weights** π_k ($\Sigma\pi_k = 1$), means μ_k and covariances Σ_k. The cluster of a point is the hidden (latent) component that generated it. EM finds the parameters $\theta = (\pi, \mu, \Sigma)$ that **maximise the log-likelihood** $\Sigma_i \log p(x_i)$.

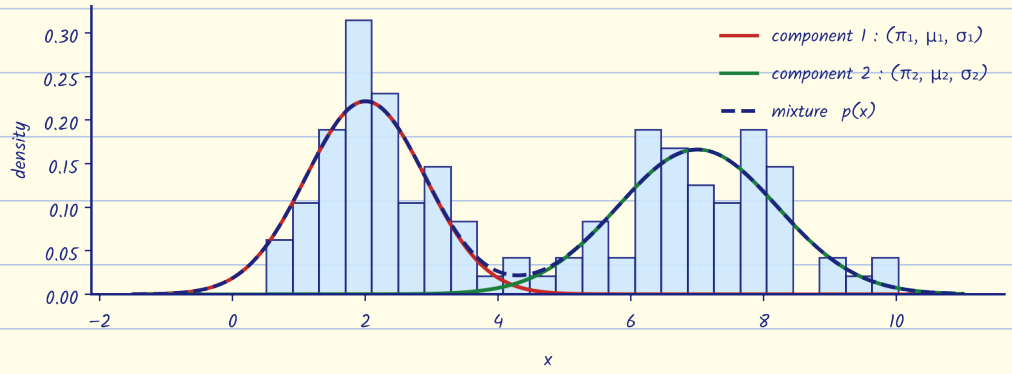


Fig 5.11 : A two-component Gaussian mixture fitted to 1-D data

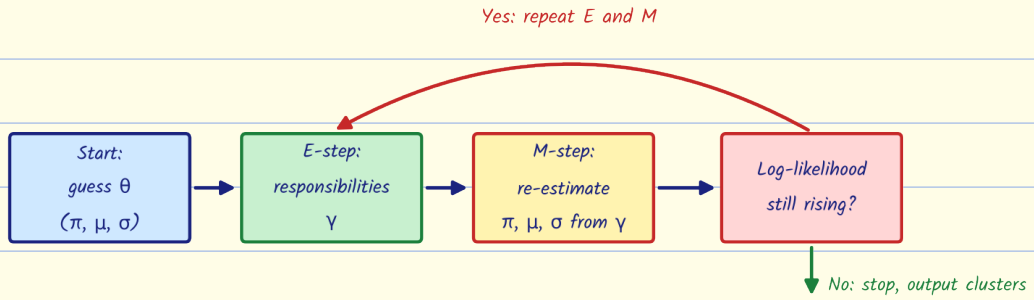


Fig 5.12 : The EM loop: E-step and M-step repeated until convergence

E-step (responsibility): $\gamma_{ik} = \pi_k \, N(x_i \mid \mu_k, \Sigma_k) \; / \; \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \Sigma_j)$

M-step: $N_k = \Sigma_i \, \gamma_{ik} \quad \mu_k = (1/N_k) \, \Sigma_i \, \gamma_{ik} \, x_i \quad \Sigma_k = (1/N_k) \, \Sigma_i \, \gamma_{ik} \, (x_i - \mu_k)(x_i - \mu_k)^T \quad \pi_k = N_k \; / \; n$

- **Algorithm:** initialise π, μ, Σ (often from K-Means) → E-step → M-step → repeat until the log-likelihood changes by less than a tolerance. Each iteration **never decreases** the likelihood.
- **Output:** soft clusters γ_{ik} (probabilities); hard label = $\text{argmax}_k \, \gamma_{ik}$. **K-Means is a special case:** equal weights, equal spherical covariances, and γ forced to 0/1.

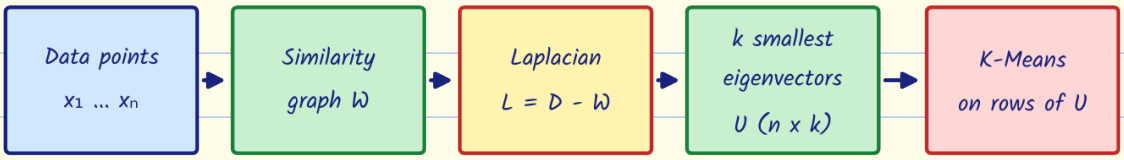
Worked example (one EM iteration). Data x = 1, 2, 5, 8, 9; K = 2; start μ = (2, 8), σ = (2, 2), π = (0.5, 0.5). E-step responsibilities γ_{i1} (cluster 1): x=1: 0.998, x=2: 0.989, x=5: 0.500, x=8: 0.011, x=9: 0.002. The middle point x = 5 is exactly 0.5 / 0.5.

Parameter	Cluster 1	Cluster 2	How
$N_k = \Sigma\gamma$	2.50	2.50	sum of responsibilities
$\pi_k = N_k/5$	0.50	0.50	share of data
μ_k	2.234	7.766	weighted mean (moved from 2 and 8)
σ_k	1.533	1.533	weighted std. deviation (from 2)
log-likelihood	-12.181 → -11.801		increased after one EM step

Point	K-Means	EM / GMM
Assignment	Hard	Soft (probabilities)
Cluster shape	Spherical, equal size	Elliptical (full covariance), different sizes
Model	Distance only	Probabilistic generative model
Model selection	Elbow, silhouette	BIC / AIC, likelihood
Weakness	Needs round clusters	Local optimum, singular covariance, slower

5.14 Spectral Clustering

Spectral clustering uses the **eigenvectors** of a graph matrix to turn the data into a space where clusters become easy, then runs K-Means there. It finds clusters of **any shape** (connected / non-convex) that K-Means cannot.



graph -> matrix -> new coordinates -> ordinary clustering

Fig 5.13 : Spectral clustering pipeline

1. **Similarity graph:** each point is a node; edge weight $W_{ij} = \exp(-\|x_i - x_j\|^2 / 2\sigma^2)$ (Gaussian), or connect only the k nearest neighbours (k-NN graph) or points within ε.
2. **Degree matrix** $D = \mathrm{diag}(d_i)$, $d_i = \sum_j W_{ij}$. **Laplacian** $L = D - W$ (normalised: $L_{sym} = I - D^{-1/2} W D^{-1/2}$).
3. Compute the **k smallest eigenvectors** of L and stack them as columns of U (n × k). (Normalised version: normalise each row of U to length 1.)
4. Treat each **row of U** as a new k-dimensional point and run **K-Means** on the rows. Point i gets the cluster of row i.

> **Why it works:** for any vector f, $f^T L f = \frac{1}{2} \sum_{ij} W_{ij} (f_i - f_j)^2 \geq 0$. Small values mean f changes only across weak edges = a **graph cut**.

- The eigenvalue **0** of L appears **once for every connected component**; with k clusters nearly separated, k eigenvalues are close to 0 (the **eigengap** also helps choose k).
- **Spectral clustering approximates the normalised graph cut** (Ncut / RatioCut), which is NP-hard to solve exactly.

Worked example: 6 nodes: triangle {1,2,3}, triangle {4,5,6}, plus one bridge edge 3-4 (all weights 1). Eigenvalues of L = D - W: 0.000, 0.438, 3.000, 3.000, 3.000, 4.562. Only the first is 0 (one connected graph) and the second is small (0.438) - a clear gap, so K = 2. The second eigenvector (Fiedler vector) is (−0.46, −0.46, −0.26, +0.26, +0.46, +0.46): its **sign** separates {1,2,3} from {4,5,6} (for K = 2 no K-Means is needed).

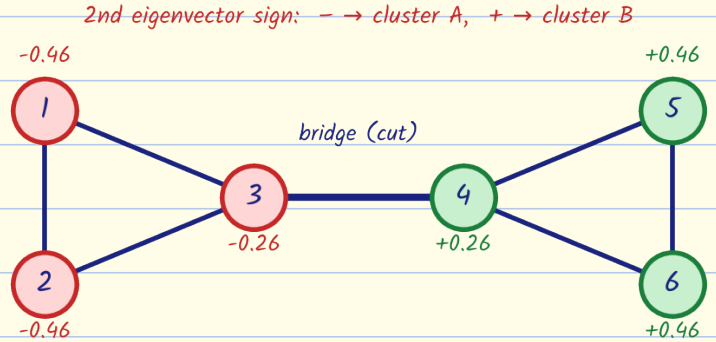


Fig 5.14 : Two triangles joined by a bridge; the 2nd Laplacian eigenvector splits them

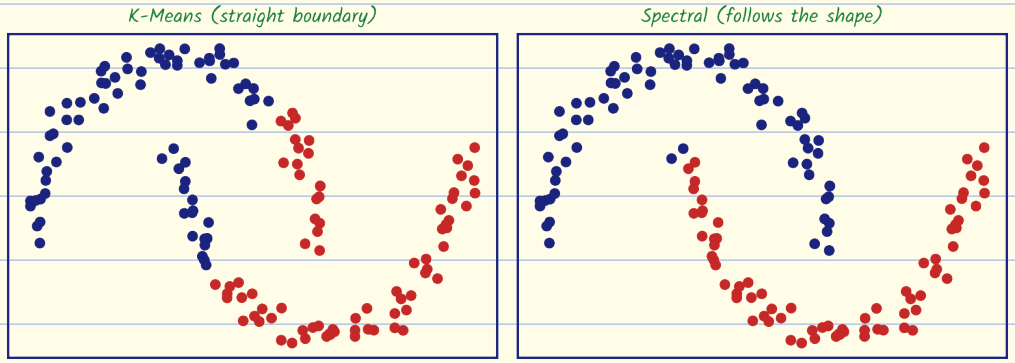


Fig 5.15 : Two half-moons: K-Means cuts straight through them, spectral clustering follows the shape

- **Pros:** arbitrary shapes, uses only a similarity matrix, strong results on images / graphs. **Cons:** $O(n^3)$ eigen-decomposition (use sparse graphs / Lanczos), $n \times n$ matrix, choice of σ or neighbours, K must be known, sensitive to graph construction.

Part V : Comparison and Revision

5.15 Comparison of Clustering Methods

Method	Type	Needs K?	Shape handled	Key idea / cost
Agglomerative	Hierarchical, hard	No	Depends on linkage	Merge closest pair; $O(n^3)$
Divisive (DIANA)	Hierarchical, hard	No	Depends on split	Split by splinter group; costly
K-Means	Partitional, hard	Yes	Round, equal size	Mean + nearest centre; $O(nKdt)$
Fuzzy C-Means	Soft (fuzzy)	Yes	Round	Membership u_{ij} weighted by m
Rough K-Means	Soft (rough)	Yes	Round	Lower / upper sets, weights w_l, w_u
EM / GMM	Soft (probabilistic)	Yes	Elliptical	Maximise likelihood; E and M steps
Spectral	Graph-based	Yes	Any (connected)	Eigenvectors of Laplacian + K-Means

* **Study tip:**
Choosing a method: need a tree or small data → agglomerative. Large data, round clusters → K-Means. Overlapping clusters → FCM / EM. Vague boundaries → Rough K-Means. Strange shapes → spectral.

Quick Revision - Unit V

- **Clustering** = unsupervised grouping; high intra-cluster, low inter-cluster similarity. Features must be scaled; pick a distance first.
- **Hard partition:** non-empty, cover X, disjoint. **Soft partition:** memberships $u_{ij} \in [0,1]$, row sum 1. Number of partitions = Stirling $S(n,K) \approx K^n/K!$.

- **Matrix factorization:** $V \approx W H$; K-Means = binary W; NMF (non-negative) gives soft, parts-based clusters.
- **Divisive** = top-down (DIANA, bisecting K-Means). **Agglomerative** = bottom-up; linkage: single (min), complete (max), average; dendrogram cut gives clusters.
- **K-Means:** $J = \Sigma\Sigma||x - \mu||^2$; assign → update; converges to a local minimum; choose K by elbow / silhouette; k-means++ init; K-Medoids for outliers.
- **FCM:** $u_{ij} = 1 / \Sigma(d_{ij}/d_{ik})^{2/(m-1)}$; $v_j = \Sigma u^m x / \Sigma u^m$. **Rough K-Means:** lower / upper approx., $c = w_l \cdot \text{mean(lower)} + w_u \cdot \text{mean(boundary)}$.
- **EM / GMM:** E-step γ_{ik}; M-step N_k, μ, Σ, π; likelihood never decreases; K-Means is the hard limit.
- **Spectral:** graph $W \rightarrow L = D - W \rightarrow k$ smallest eigenvectors → K-Means on rows; finds non-convex clusters; eigenvalue 0 count = components.

Likely exam questions

1. Define clustering. Explain the different types of clustering with a neat diagram and give applications.
2. What is partitioning of data? Differentiate hard and soft partitions with the conditions each must satisfy.
3. Explain matrix factorization for clustering (K-Means and NMF as $V \approx WH$) with an example.
4. Explain divisive and agglomerative hierarchical clustering. Cluster the points 1, 2, 6, 8, 15 by single linkage and draw the dendrogram.
5. Describe the K-Means algorithm. Apply two iterations on a small data set and discuss how K is chosen, its advantages and limitations.
6. Explain the Fuzzy C-Means algorithm with the membership and centre update equations. How does it differ from K-Means?
7. What are rough sets? Explain the Rough K-Means algorithm with lower and upper approximations.
8. Explain EM-based clustering using Gaussian mixtures (E-step and M-step). Explain spectral clustering and where it is better than K-Means.